Code Walkthrough Report

Project: Three.js Gift Box Lab

Project Structure

File	Responsibility
src/main.jsx	Entry point. Mounts the React app into the HTML page.
src/App.jsx	Core file. Owns all state, sets up the 3D scene once, and wires up interactivity and animations.
src/ConfigPanel.jsx	UI sidebar, text input and colour pickers. Receives values and callbacks from App.jsx via props.
src/three/scene.js	Creates the Three.js scene, camera, renderer, lighting, ground plane, and orbit controls.
src/three/giftBox.js	Builds all 3D geometry -- box walls, lid, ribbons, bow knot, and the clickable button. Returns material references and the pivot group.
src/three/textMesh.js	Builds the 3D greeting text and returns the mesh and material so App.jsx can animate and swap it.

Explanation of Key Concepts and Logic

Section 1 - Converting Mouse Position to 3D Space

File: src/App.jsx

Problem:

The browser gives you mouse coordinates in pixels (e.g., clientX: 742, clientY: 310). Three.js does not understand pixels. It works in a coordinate space called NDC (Normalized Device Coordinates) where X and Y both range from -1 to +1, with (0, 0) at the exact center of the canvas.

Code:

```
const toNDC = (e) => {
      const r = mount.getBoundingClientRect();
      ptr.x = ((e.clientX - r.left) / r.width) * 2 - 1;
      ptr.y = -((e.clientY - r.top) / r.height) * 2 + 1;
    };
```

Explanation:

Step	Code	What it does
1	e.clientX - r.left	Converts viewport X to canvas-local X (removes the sidebar/offset)
2	/ r.width	Normalizes to [0, 1]
3	* 2 - 1	Shifts to [-1, +1], so left edge = -1, right edge = +1

Step	Code	What it does
4	Negative sign on Y	Browser Y goes DOWN, 3D Y goes UP. The negative flips the axis.

The resulting ptr vector is fed directly into raycaster.setFromCamera(ptr, camera), which is explained in the next section.

Section 2 - How Raycaster Detects What You Clicked

File: src/App.jsx

Problem :

The 3D scene is a flat image on your screen. When you click, there is no automatic way to know which 3D object your mouse is over. The raycaster solves this by firing an invisible ray from the camera through the pixel you clicked, and checking which 3D objects it hits.

Code :

```
const raycaster = new THREE.Raycaster();
...
const onPointerMove = (e) => {
  toNDC(e);
  raycaster.setFromCamera(ptr, camera);

  const btnHit = raycaster.intersectObject(button3D).length > 0;

```

The same pattern repeats in onPointerDown and onPointerUp. On pointerUp, if the ray hits the button, toggle() is called. This approach is used because Three.js objects are not DOM elements, you cannot attach a regular onClick to them.

Section 3 - Making the Lid Open Like a Real Hinge

File: src/three/giftBox.js

Problem :

A box lid rotates around its back edge, not its center. If you just rotate the lid mesh around its own center, it will swing weirdly half of it would go into the box. The solution is to create an invisible Group object whose position is exactly at the back edge, then make the lid a child of that group.

Code :

```
const pivot = new THREE.Group();
pivot.position.set(0, 0.75, -1.03);
scene.add(pivot);

const lidMesh = new THREE.Mesh(
  new THREE.BoxGeometry(LID_W, LID_H, LID_D),
  lidMat,
);
lidMesh.position.set(0, LID_H / 2, LID_D / 2);
lidMesh.castShadow = true;
pivot.add(lidMesh);

```

When GSAP animates pivot.rotation.x, it rotates the entire group around the pivot's local X axis. Because lidMesh is a child positioned in front of the pivot, it swings open exactly like a physical lid. The value -Math.PI * 0.78 (roughly 140 degrees) is tuned to produce a convincing open position without the lid clipping through the box.

Section 4 - Centering the 3D Text

File: src/three/textMesh.js

Problem :

When Three.js generates 3D text geometry, it starts drawing from the bottom-left corner. If you place it at position (0, 0, 0), it will be misaligned to the right. The text needs to be shifted so its center aligns with the world origin to sit neatly above the box.

Code :

```
const geo = new TextGeometry(text, {
  font,
  size: 0.3,
  depth: 0.1,
  curveSegments: 8,
  bevelEnabled: true,
  bevelThickness: 0.018,
  bevelSize: 0.012,
  bevelSegments: 4,
});
geo.computeBoundingBox();
const { min, max } = geo.boundingBox;
geo.translate(-(max.x + min.x) / 2, 0, -(max.z + min.z) / 2);

const mat = new THREE.MeshStandardMaterial({
  color,
  metalness: 0.7,
  roughness: 0.15,
});
const mesh = new THREE.Mesh(geo, mat);
mesh.position.set(0, 0.85, 0);

```

Math :

```
min.x = 0       (text starts at left edge)
max.x = 1.4     (text ends at right edge, depends on string length)
```

```
center = (0 + 1.4) / 2 = 0.7
offset = -0.7   //shift geometry left by this amount

Result: text now spans [-0.7, +0.7], centered at 0
```

This is done with geo.translate() rather than mesh.position.x because the mesh's position is already being animated by GSAP. Baking the centering into the geometry itself avoids any conflict.

Section 5 - The Open/Close Animation with Interruption Handling (toggle)

File: src/App.jsx

Problem :

The open and close sequences are multi-step animations. The user could click the button before an animation finishes (mid-open, mid-close), so the function must:

- Kill whatever is currently running
- Read the freshest mesh reference (text may have been rebuilt)
- Run the correct animation sequence based on current state
- Chain the animation steps with controlled overlap

Code :

```jsx
const toggle = () => {
  const textMesh = textRef.current;
  if (activeTlRef.current) activeTlRef.current.kill();
  const tl = gsap.timeline({
    onComplete: () => {
      activeTlRef.current = null;
    },
  });
  activeTlRef.current = tl;

  if (!isOpenRef.current) {
    isOpenRef.current = true;
    tl.to(pivot.rotation, {
      x: -Math.PI * 0.78,
      duration: 1.0,
      ease: "power2.out",
    })
      .to(
        textMesh.position,
        { y: 1.55, duration: 0.7, ease: "power2.out" },
        "-=0.2",
      )
      .to(
        textMesh.scale,
        { x: 1, y: 1, z: 1, duration: 0.7, ease: "back.out(1.7)" },
        "<",
      );
  } else {
    isOpenRef.current = false;
    tl.to(textMesh.scale, {
      x: 0.001,
      y: 0.001,
      z: 0.001,
      duration: 0.4,
      ease: "back.in(2)",
    })
      .to(
        textMesh.position,
        { y: 0.85, duration: 0.4, ease: "power2.in" },
        "<",
      )
      .to(
        pivot.rotation,
        { x: 0, duration: 1.0, ease: "power2.inOut" },
        "+=0.05",
      );
  }
};
```

GSAP Timeline Position Markers

Marker	Meaning
(nothing)	Starts after the previous step ends
"-=0.2"	Starts 0.2 seconds before the previous step ends (overlap)
"<"	Starts exactly when the previous step started (run together)

Marker	Meaning
"+= 0.05"	Starts 0.05 seconds after the previous step ends (short delay)